Device Modeling and Architectures

@172.16.1.72

Device Modeling and Architectures

Model: a set of functional objects and rules for composing these objects

Architecture: a set of implementation components and their connections

Models

- State-oriented models
  - Finite-state machine (FSM), Petri net, Hierarchical concurrent FSM
- Activity-oriented models
  - Data flow graph, Flowchart
- Structure-oriented models
  - Block diagram, RT netlist, Gate netlist
- Data-oriented models
  - Entity-relationship diagram, Jackson's diagram
- Heterogeneous models
  - Control/data flow graph, Structure chart, Programming language paradigm, Object-oriented paradigm, Program-state machine, Queueing model

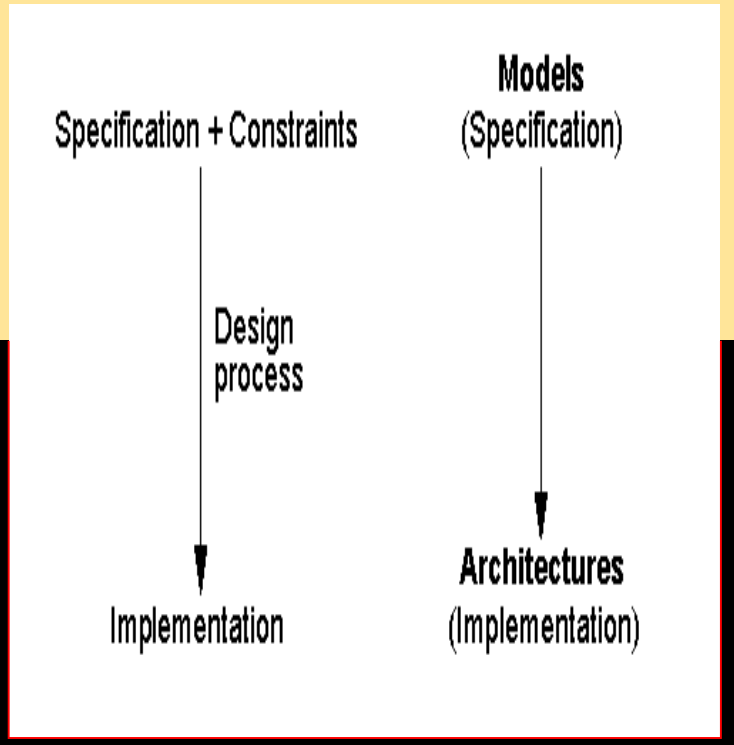
Device Modeling and Architectures

- First Step
  - Precisely specify the functionality

    using set of physical components
  - Precision by thinking system as a collection of simple subsystems or models
  - There are at least FIVE methods of decomposing the functionality into simple subsystems

Device Modeling: Qualities

- Formal
  - To eliminate any ambiguity
- Complete



Level of Abstractions in System Design

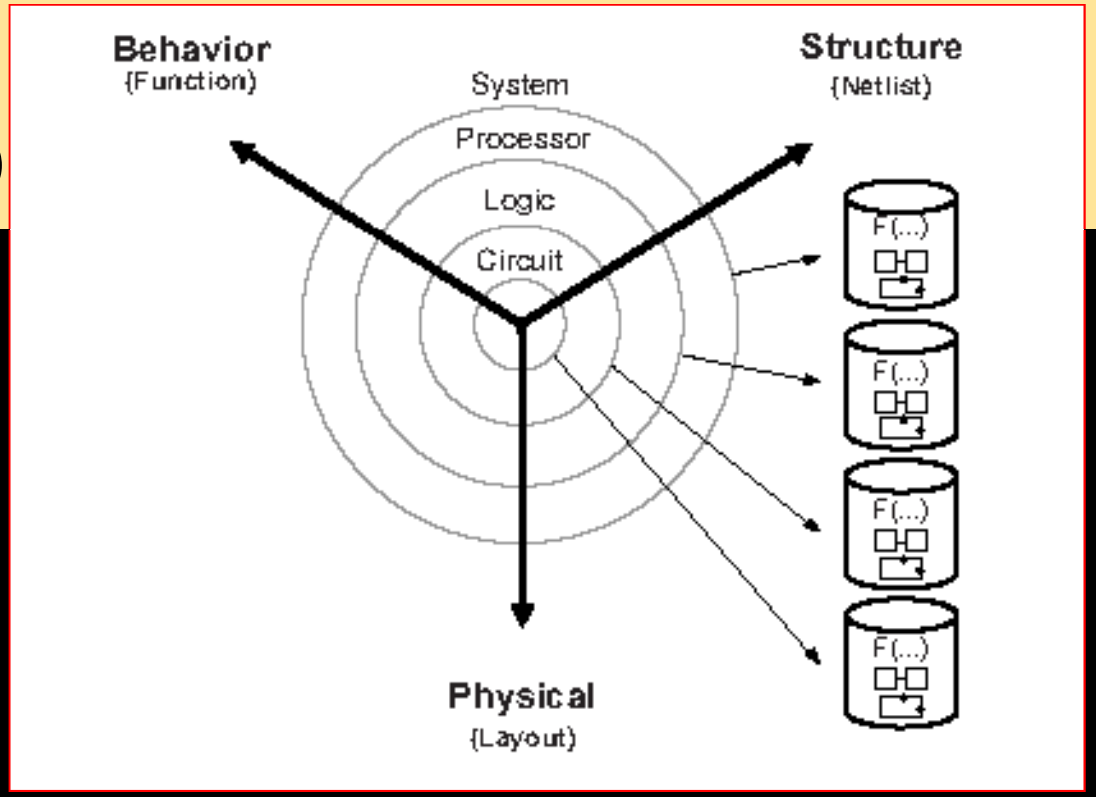
Co-simulation of software (SW) and hardware (HW) components

Three aspects of every design

Top-down

– behavior

(called as functionality)



Device Modeling: State Oriented

PROCESSOR-LEVEL BEHAVIORAL MODEL

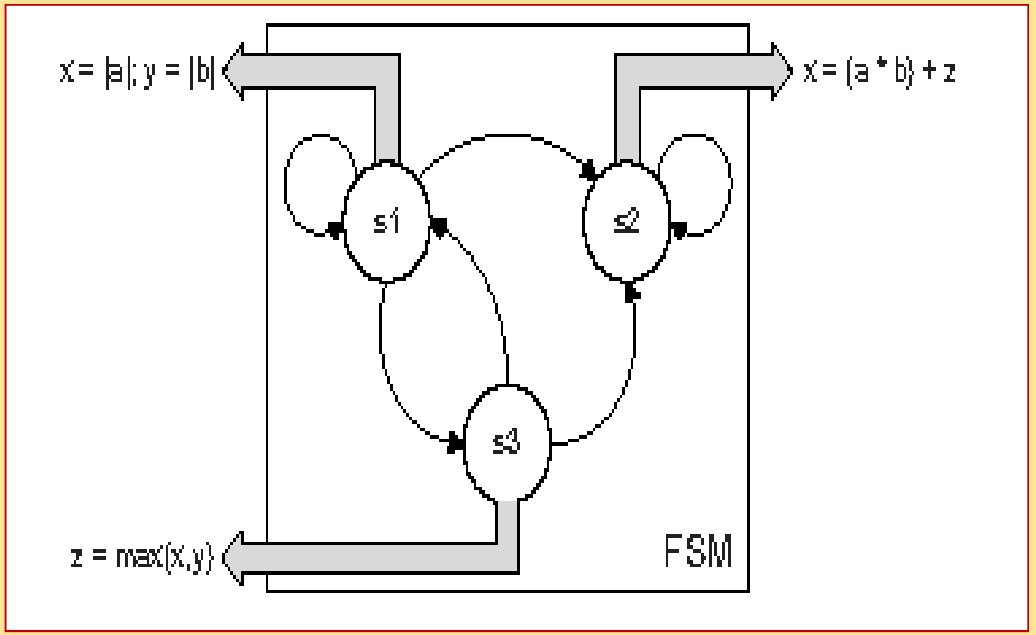
FSM with Data: Set of states and a set of transitions from state to state, which are taken when some input variables reach the required value

FSMD with three states, *S1, S2, and S3, and with arcs representing state changes under different inputs*

Each state executes a computation represented by one or more arithmetic expressions

*Not clock-accurate since computation in each state may take more than one clock cycle.

computes two functions, *x = |a| and y = |b|, and in state S3 it computes the* function *z = max (x, y).*



FSMD

State oriented: Finite-state machine (Mealy model)

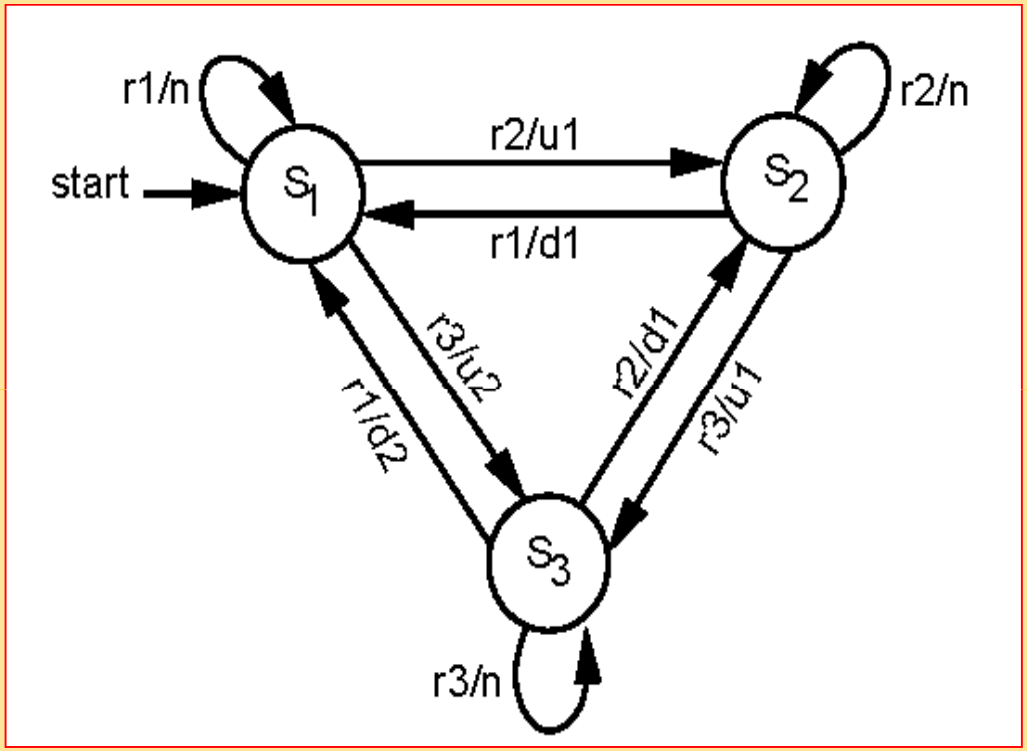
S = { s1, s2, s3}

I = {r1, r2, r3}

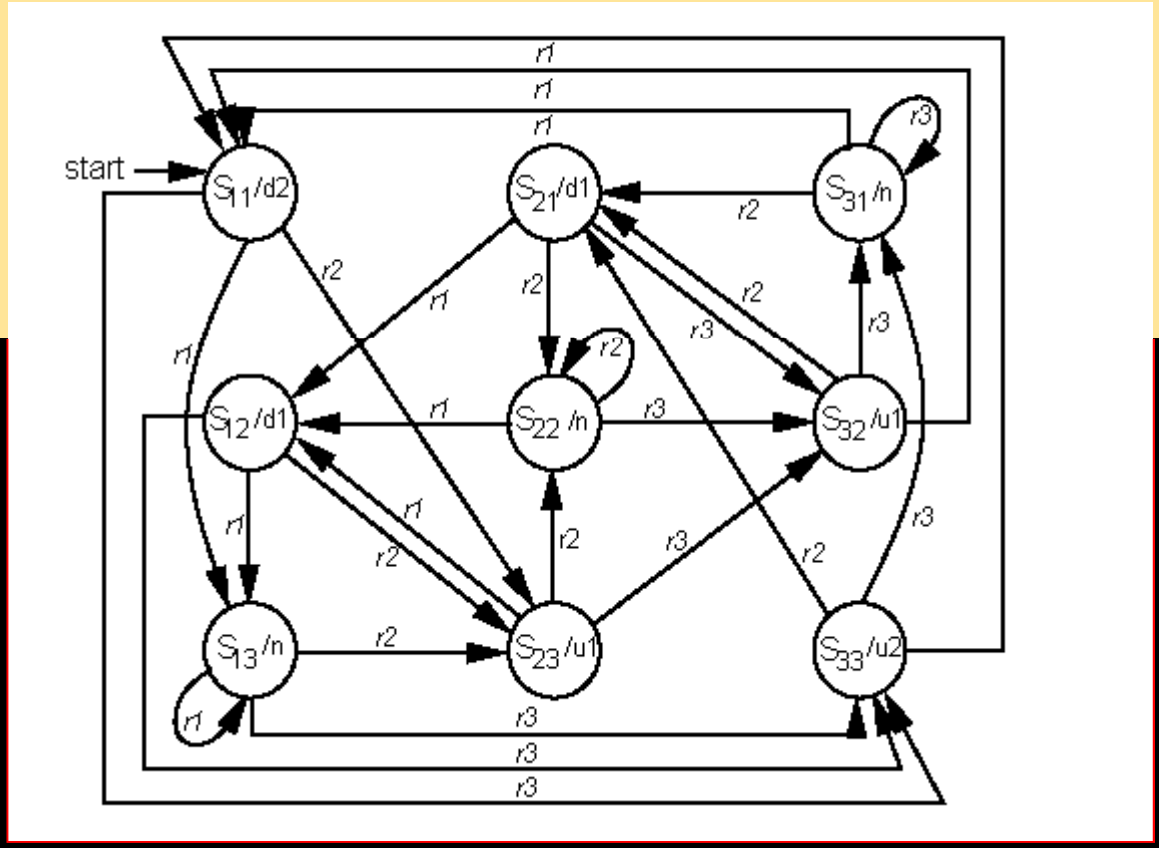
O = {d2, d1, n, u1, u2}

f: S x I –> S

h: S x I –> O



State oriented: Finite-state machine (Moore model)

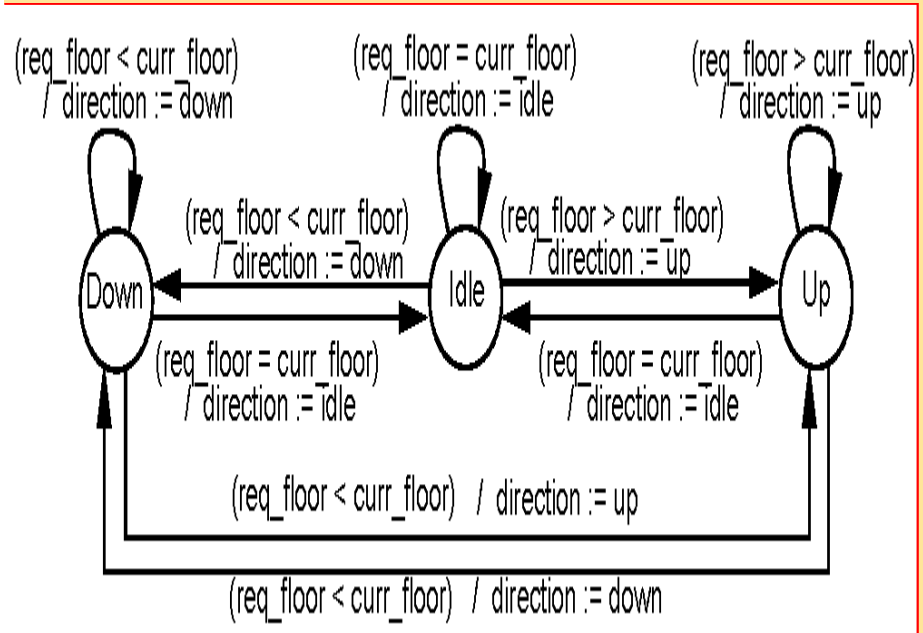


Models of an elevator controller

"If the elevator is stationary and the floor requested is equal to the current floor, then the elevator remains idle.

If the elevator is stationary and the floor requested is less than the current floor, then lower the elevator to the requested floor.

If the elevator is stationary and the floor requested is greater than the current floor, then raise the elevator to the requested floor."




```
loop
    if (req_floor = curr_floor) then
      direction := idle;
    elsif (req_floor < curr_floor) then
      direction := down;
    elsif  (req_floor > curr_floor) then
      direction := up;
    end if;
end loop;
```

Finite State Machines

- Merits:

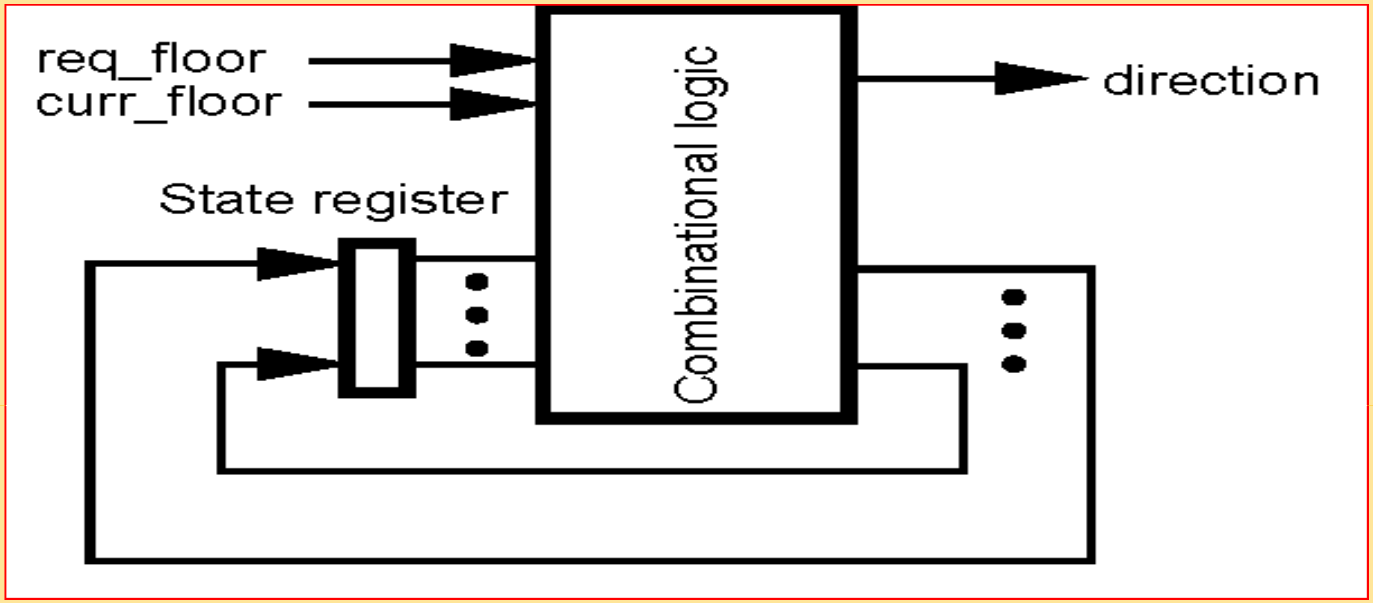
  Represent system's temporal behavior explicitly suitable for control-dominated system

- Demerits:

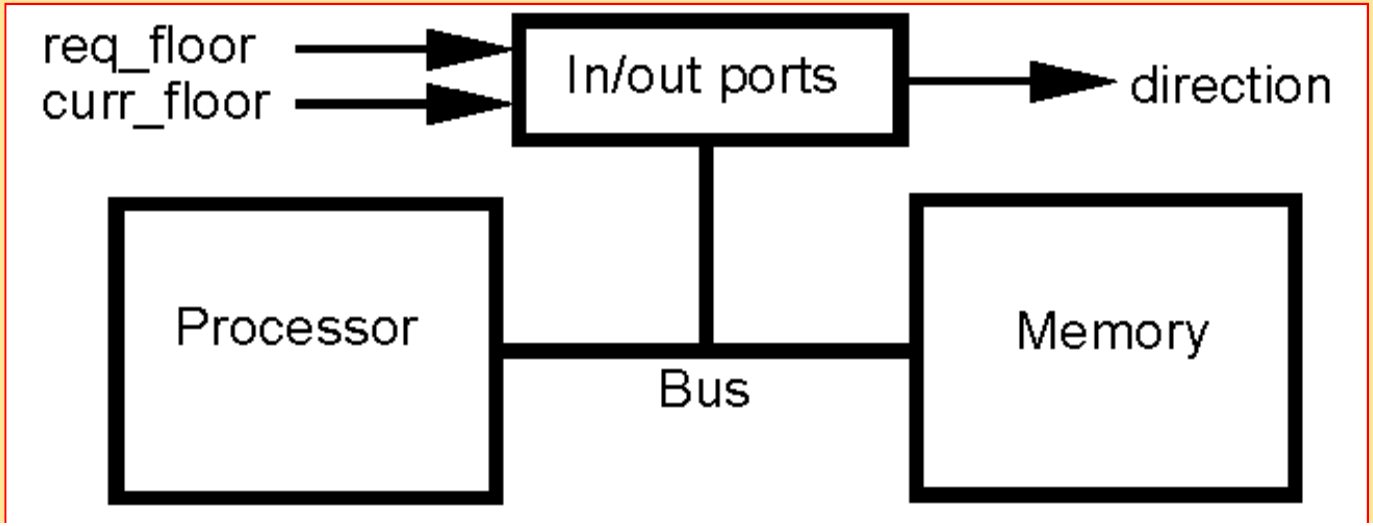
  Lack of hierarchy and concurrency resulting in state or arc explosion when representing complex systems

Architectures for Implementing the Elevator Controller

RTL Level



System Level



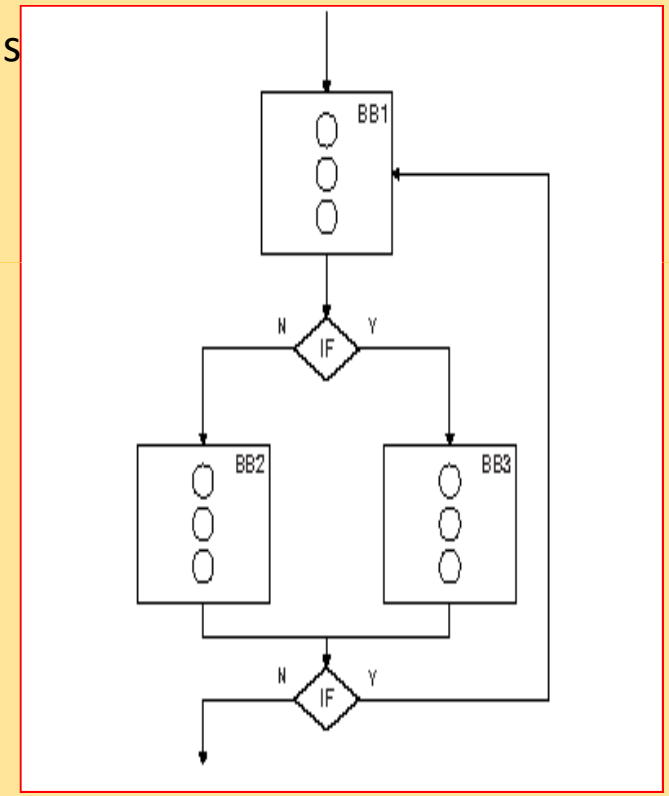
Control-Data Flow Graph (CDFG)

Consisting of ***if diamonds***, which represent if conditions

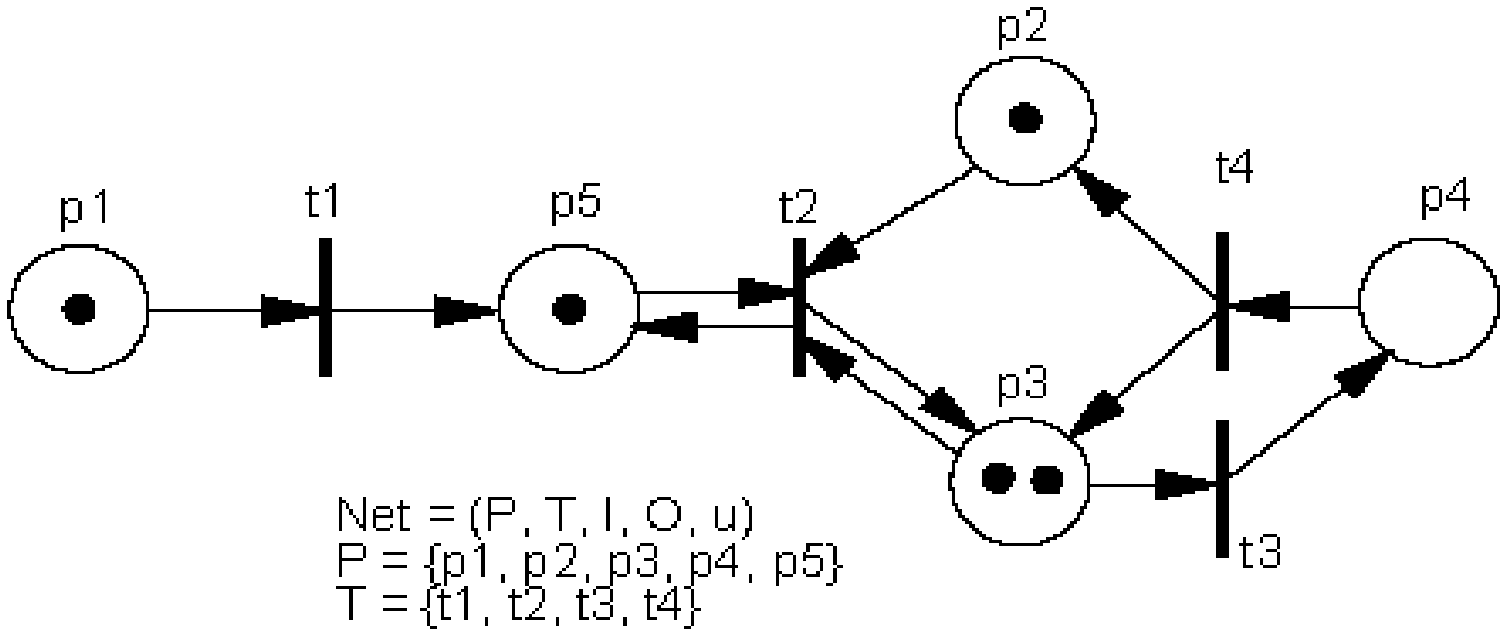
BB blocks, which represent computation

Explicitly provide the **control dependencies** between **loop** statements, ***if*** statements, and **BBs**, as well as the **data dependences** among operations inside a BB.

CDFG can be considered to be a **FSMD with superstates**, which require multiple clock cycles to execute.
Each state in such a FSMD may need **several clock cycles** to execute its assigned **BB** or ***if condition.***



State oriented: Petri nets

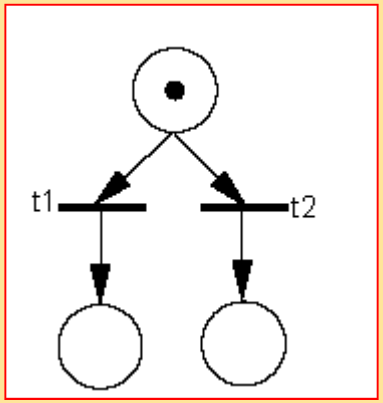


I: I(t1) = {p1}
I(t2) = {p2,p3,p5}
I(t3) = {p3}
I(t4) = {p4}

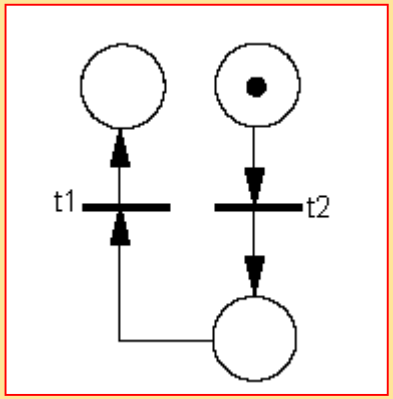
O: O(t1) = {p5}
O(t2) = {p3,p5}
O(t3) = {p4}
O(t4) = {p2,p3}

u: u(p1) = 1
u(p2) = 1
u(p3) = 2
u(p4) = 0
u(p5) = 1

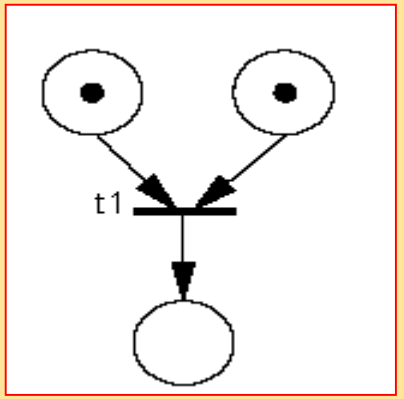
Petri Nets



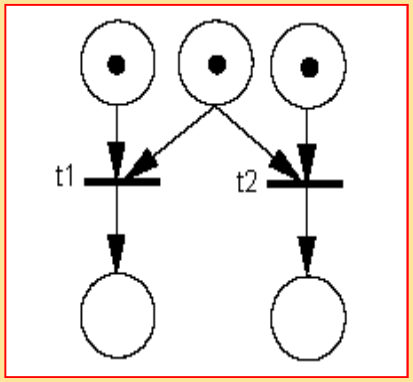
Sequence



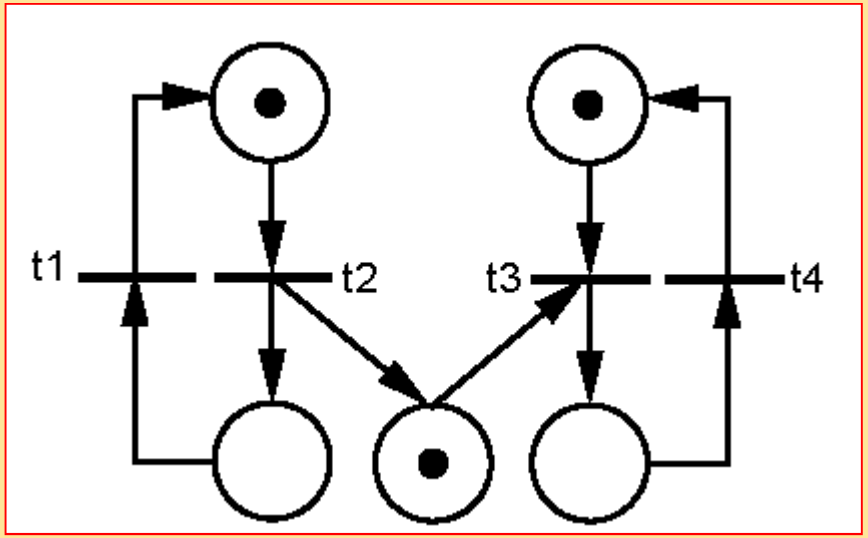
Branch



Synchronization



Resource contention



Concurrency

Petri Nets

- Merits:

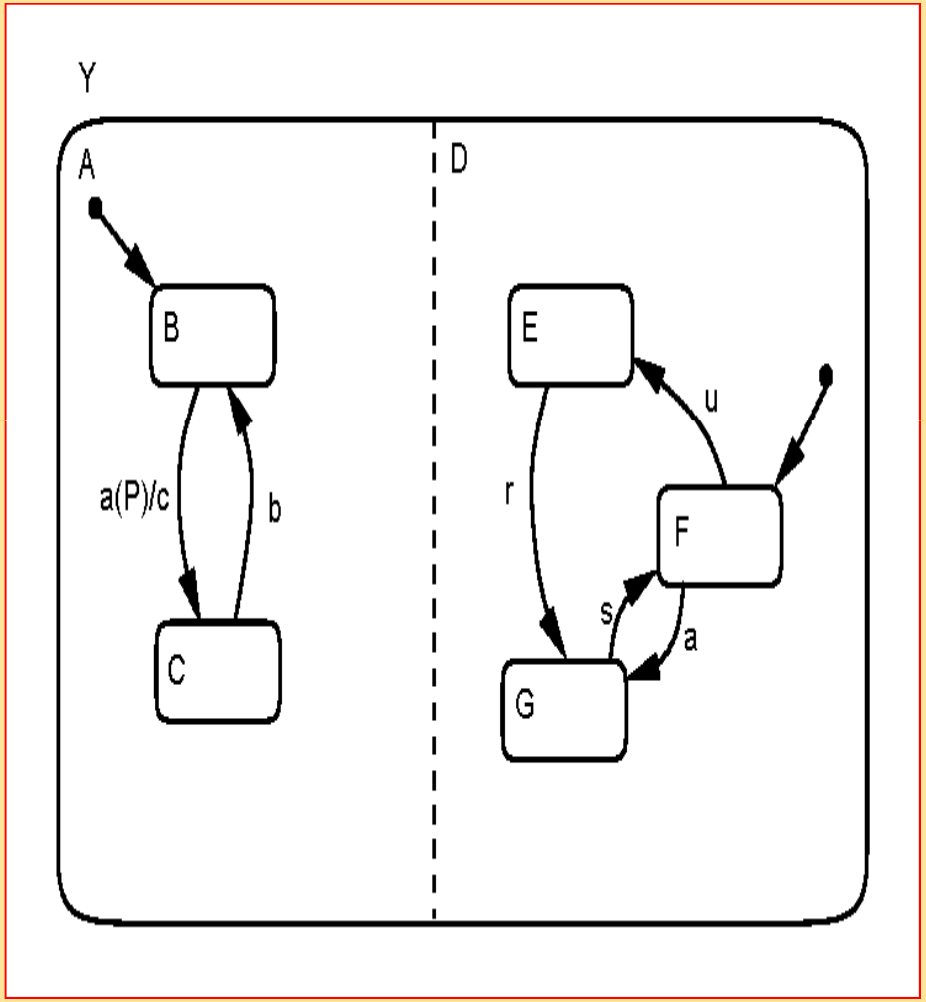
  Good at modeling and analyzing concurrent systems
- Demerits:

  ‘ at’ model that is incomprehensible when system complexity increases

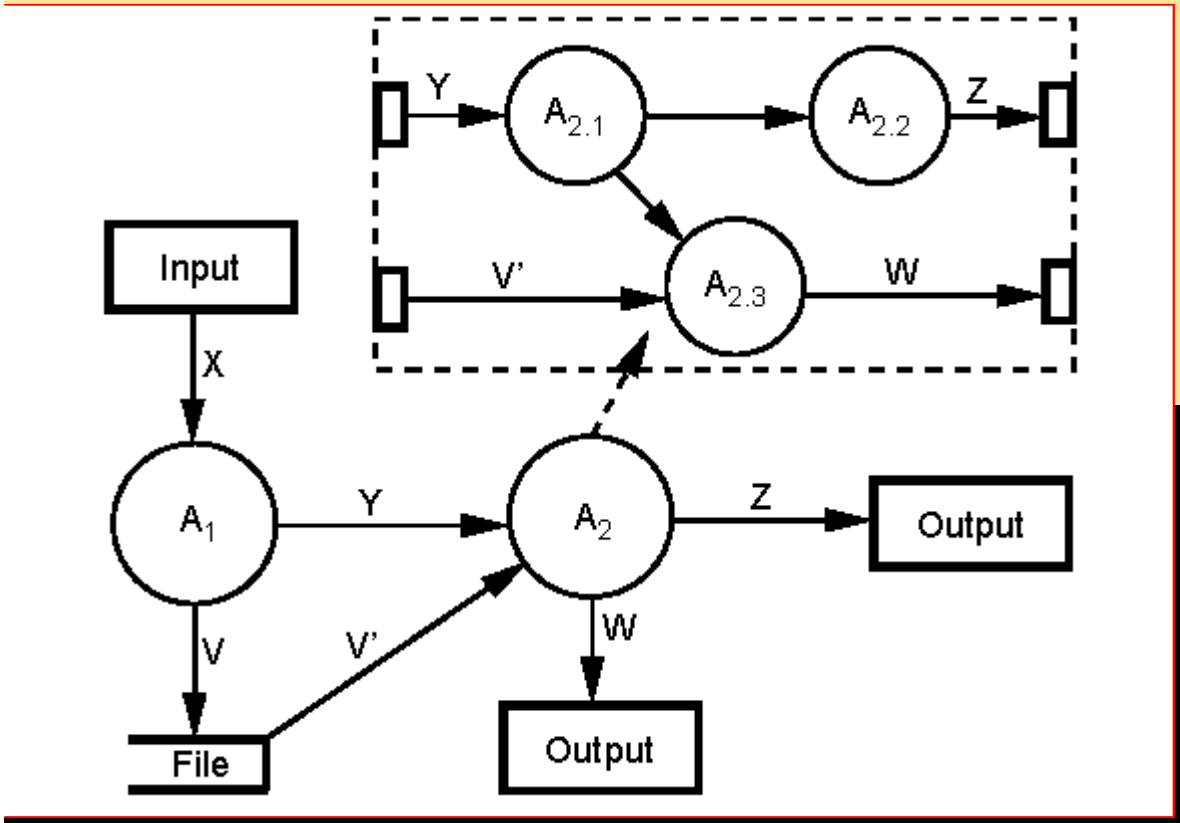
State oriented: Hierarchical concurrent FSM

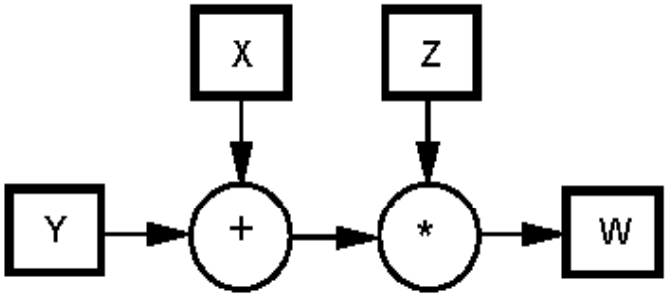
Merits:
Support both hierarchy and concurrency
good for representing complex systems

Demerits:
Concentrate only on modeling control aspects and not data and activities



Activity oriented: Data flow graphs (DFG)





Data flow graphs (DFG)

- Merits:

  Support hierarchy suitable for specifying complex transformational systems represent problem-inherent data dependencies

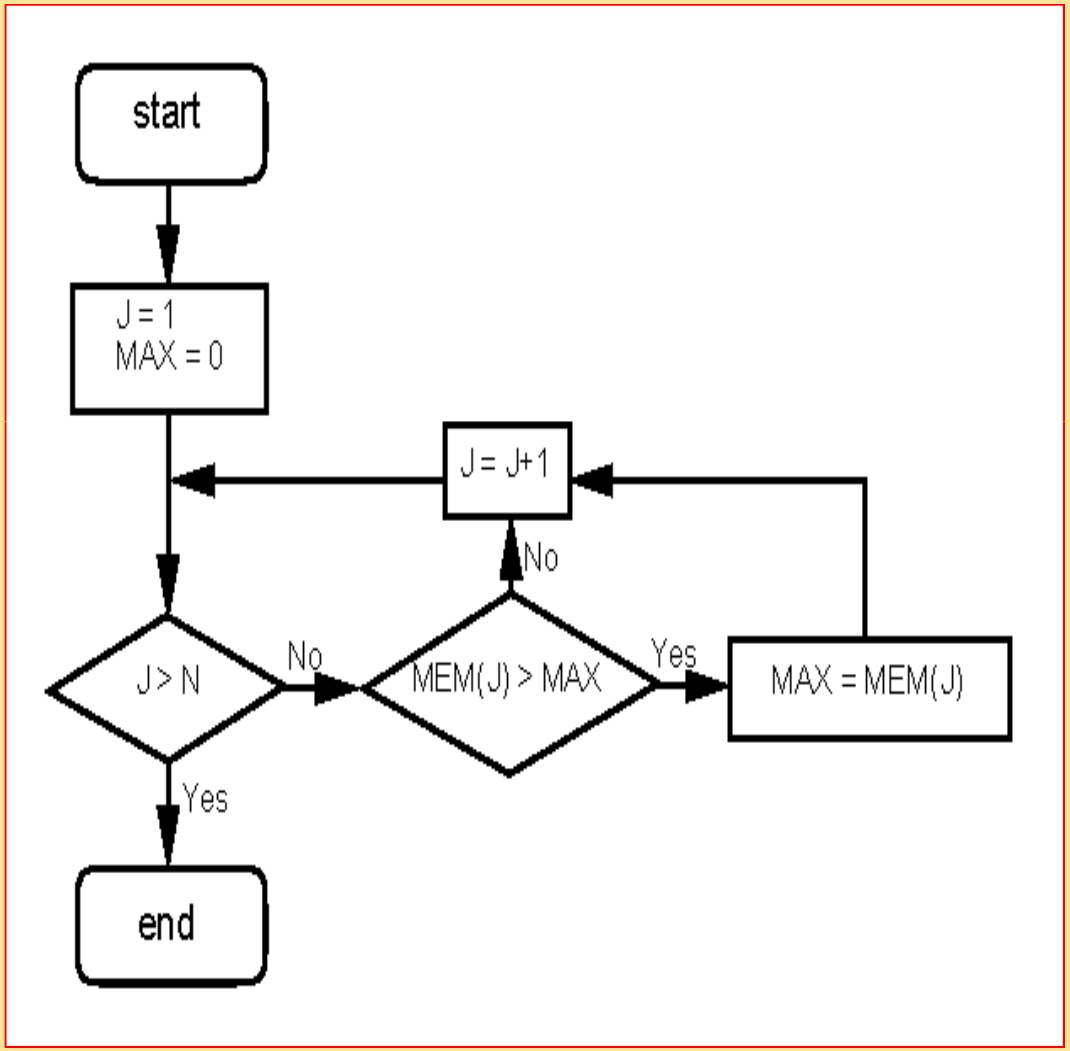
- Demerits:

  Do not express temporal behaviors or control sequencing weak for modelling embedded systems

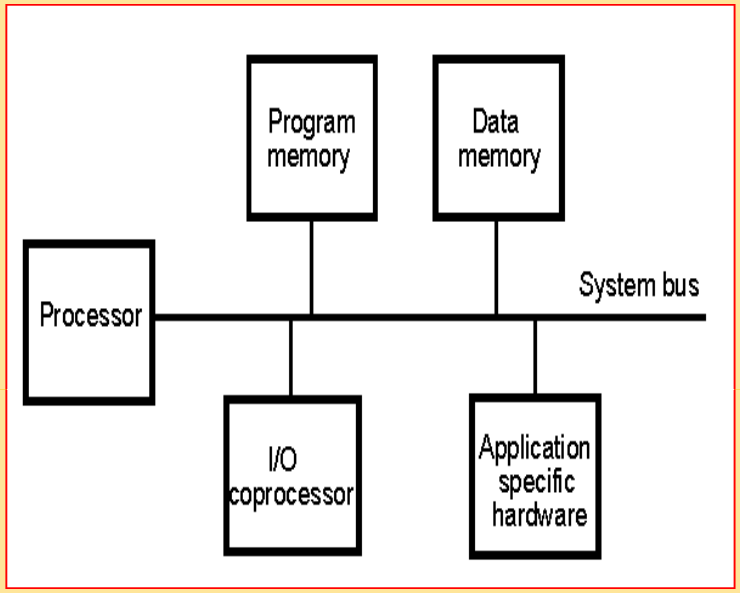
Activity oriented: Flowchart (CFG)

Merits:
useful to represent tasks governed by control flow
can impose a order to supersede natural data dependencies

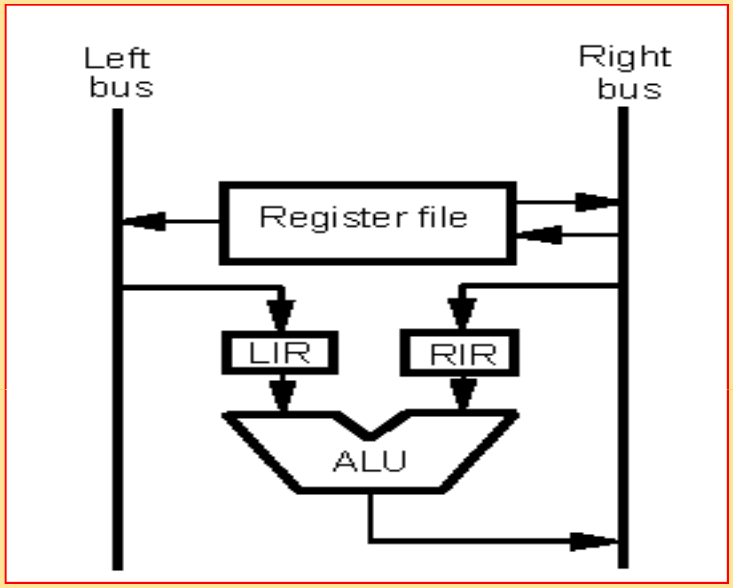
Characteristics:
used only when the system's computation is well known



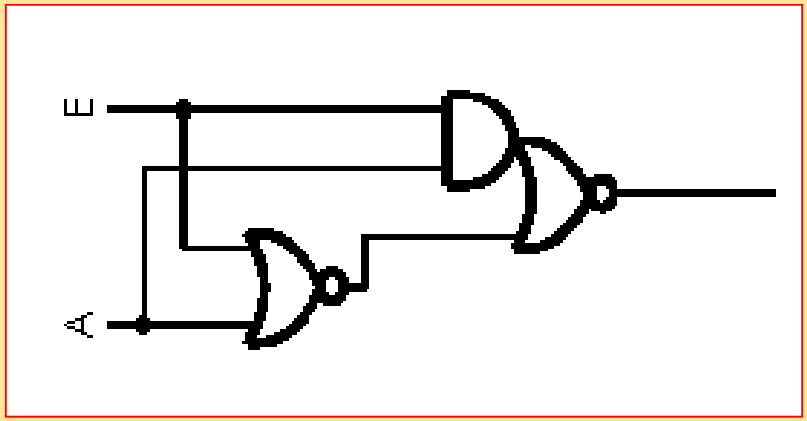
Structure oriented: Component-connectivity diagrams



BLOCK DIAGRAM



RT NET LIST

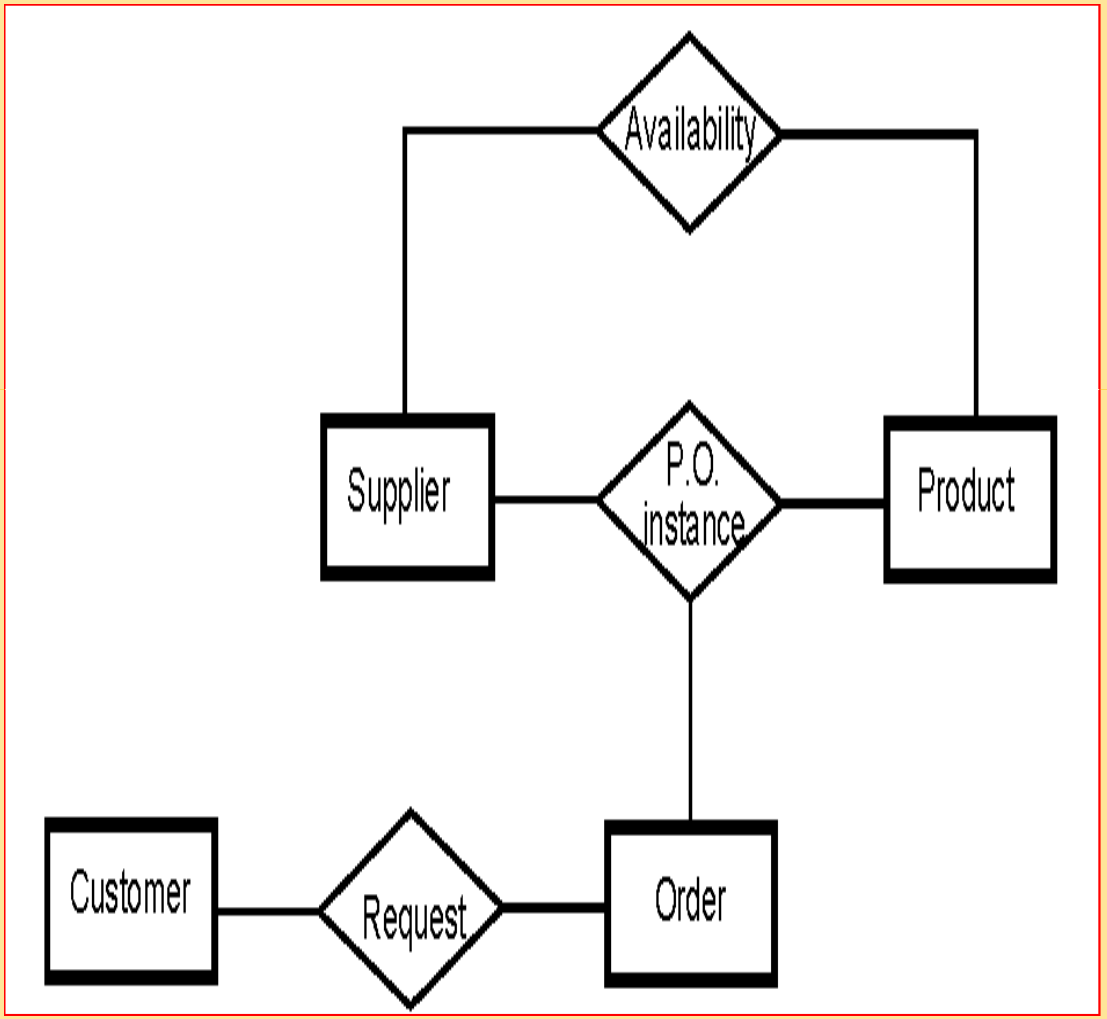


GATE NET LIST

Data oriented: Entity-relationship diagram

Merits:
provide a good view of the data in the system, also suitable for expressing complex relations among various kinds of data

Demerits:
do not describe any functional or temporal behavior of the system.



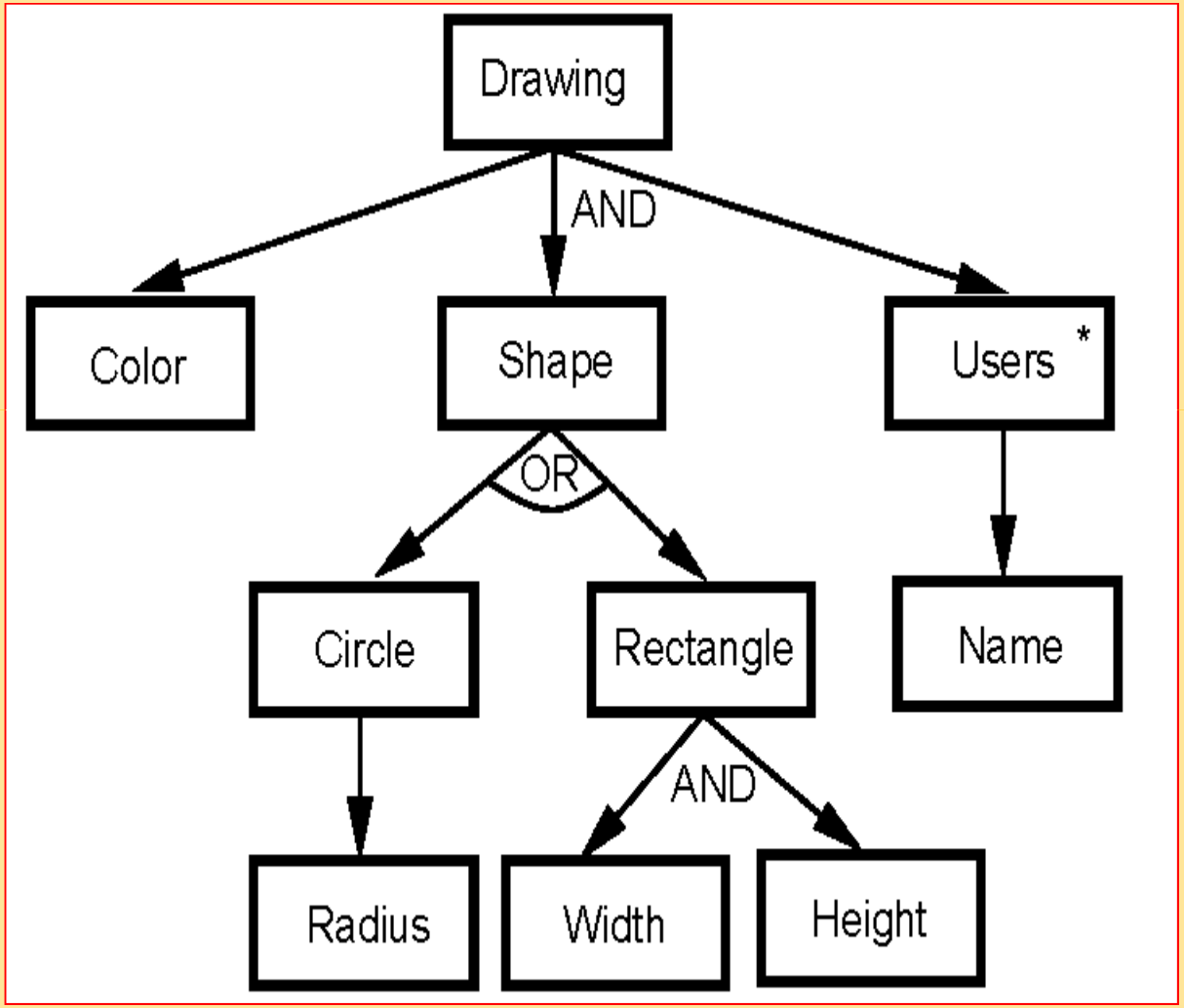
Data oriented: Jackson's diagram

Merits:

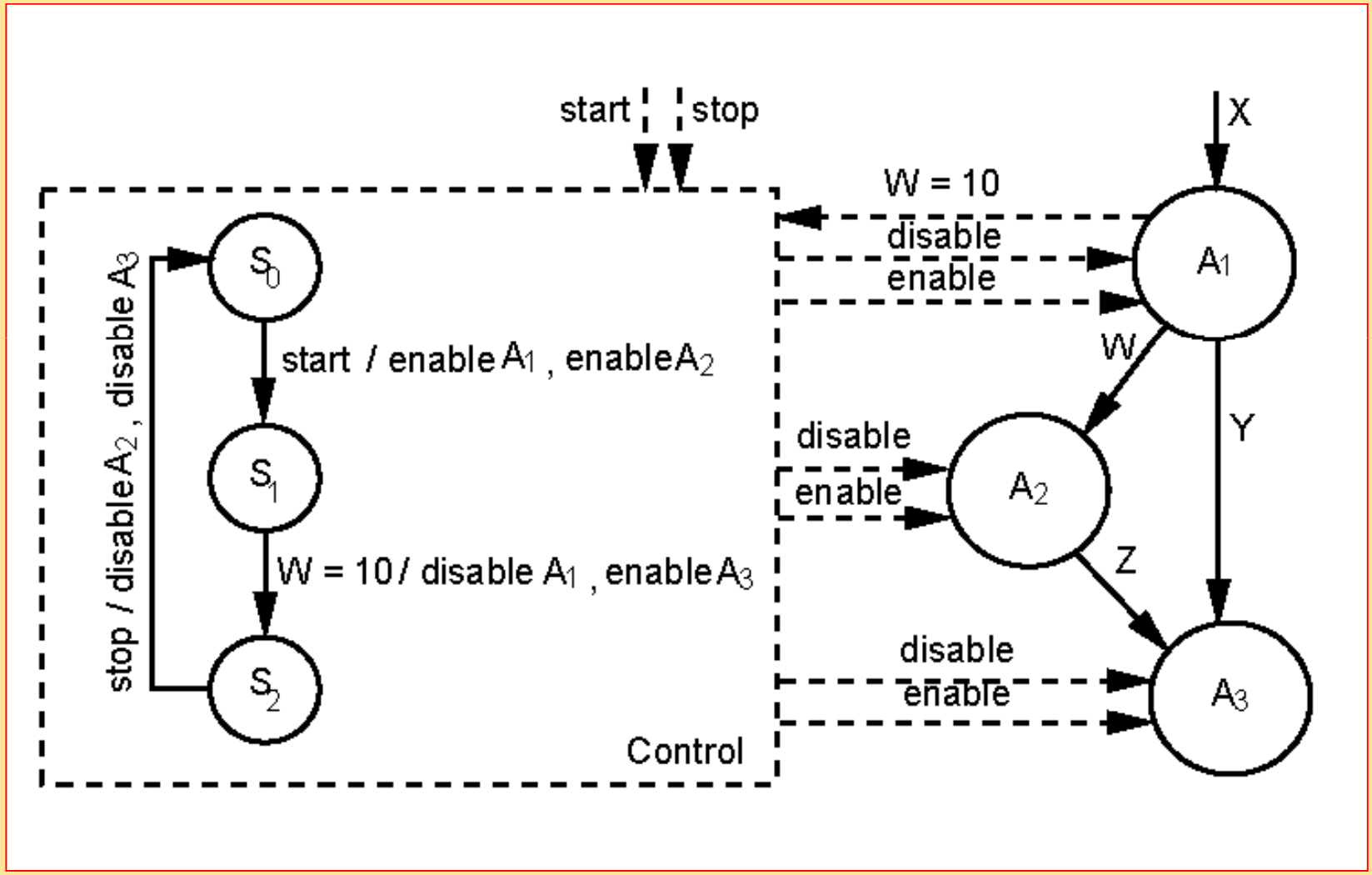
Suitable for representing data having a complex composite structure

Demerits:

do not describe any functional or temporal behavior of the system



Heterogeneous: Control/data flow graph



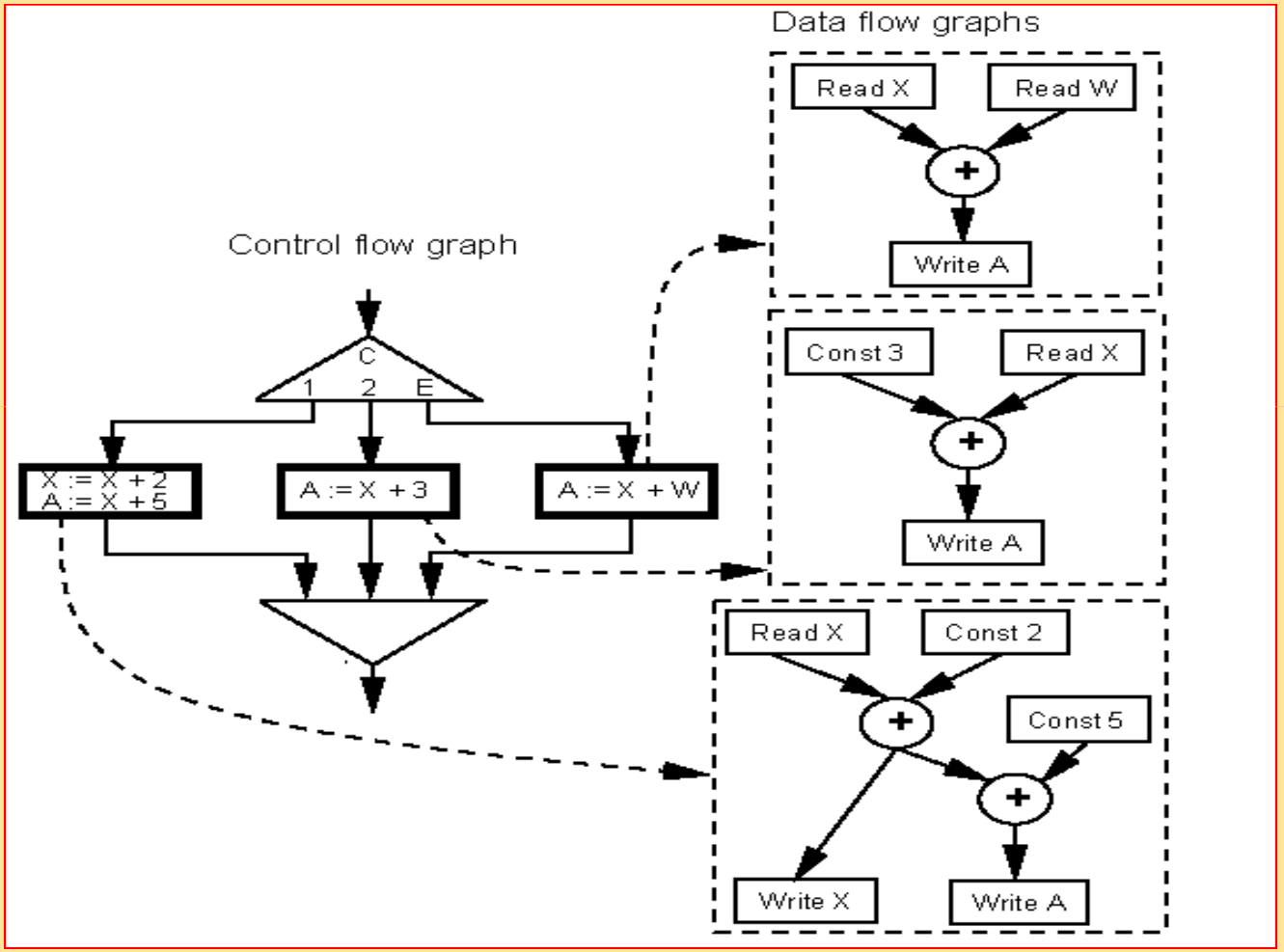
Activity Level

Heterogeneous: Control/data flow graph

```
case C is
when 1=> X=X+2;
          A=X+5;
when 2 => A= X+3;
when other = > A=X+W;
end case;
```

- Correct the inability of DFG in representing the control of a system

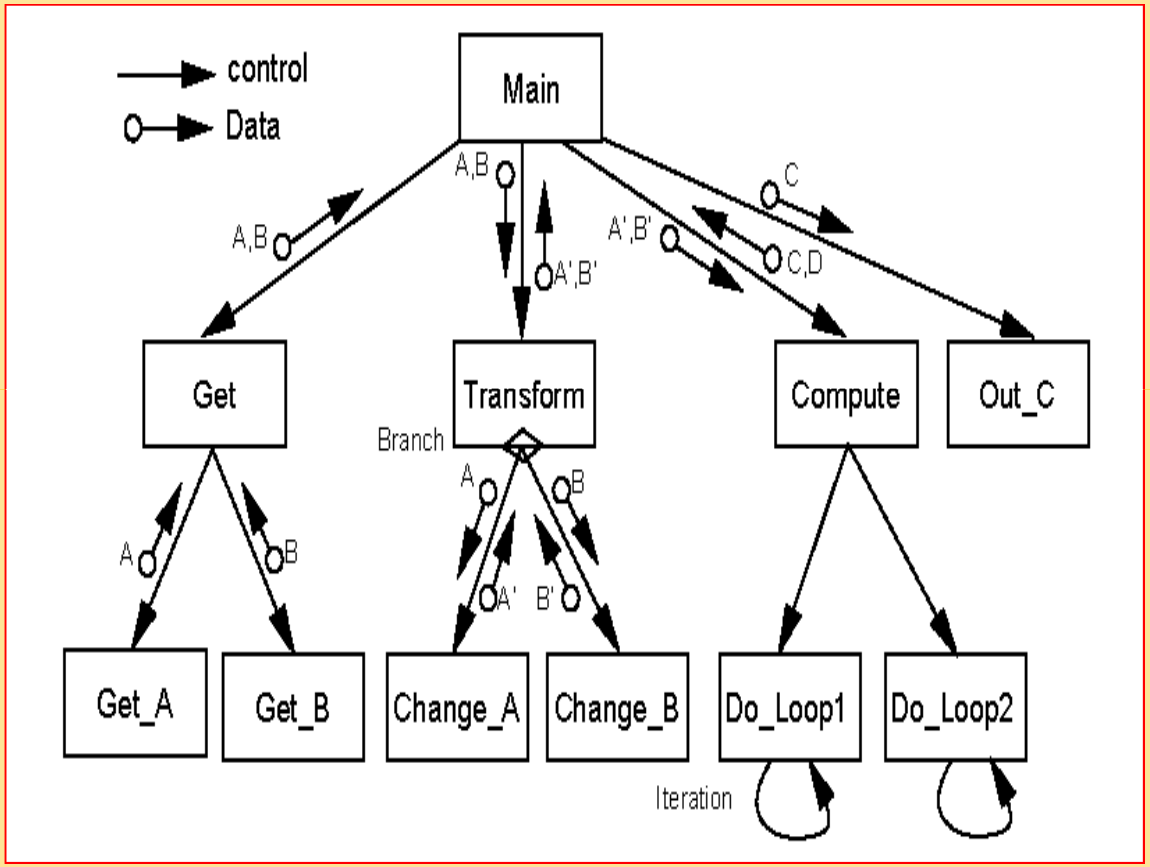
- Correct the inability of CFG to represent data dependencies



Operation Level

Heterogeneous: Structure chart

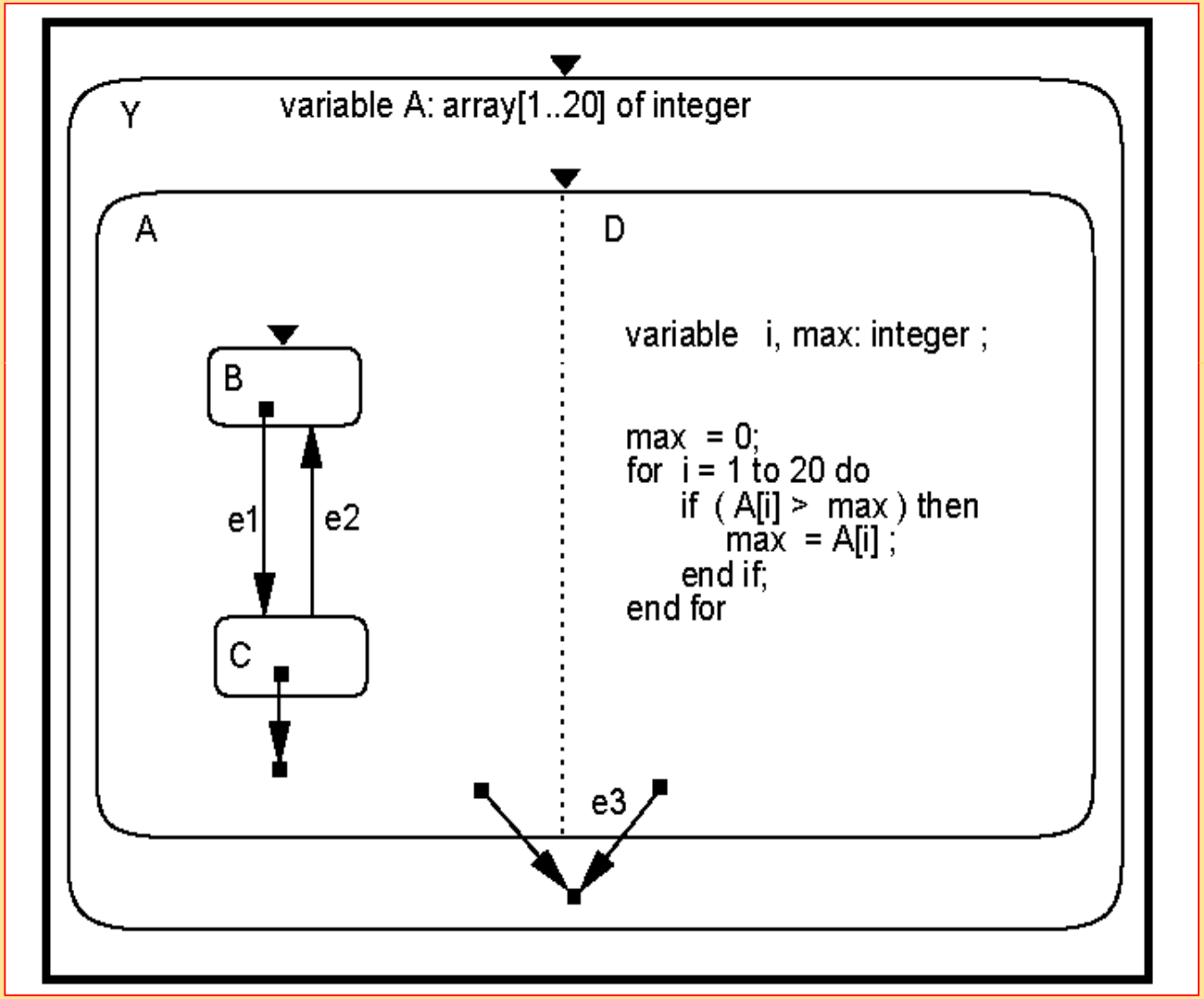
Nodes represent activities
Edges represent functions
Data passes between activity are also presented



Represent both data and control, used in the preliminary stages of program design

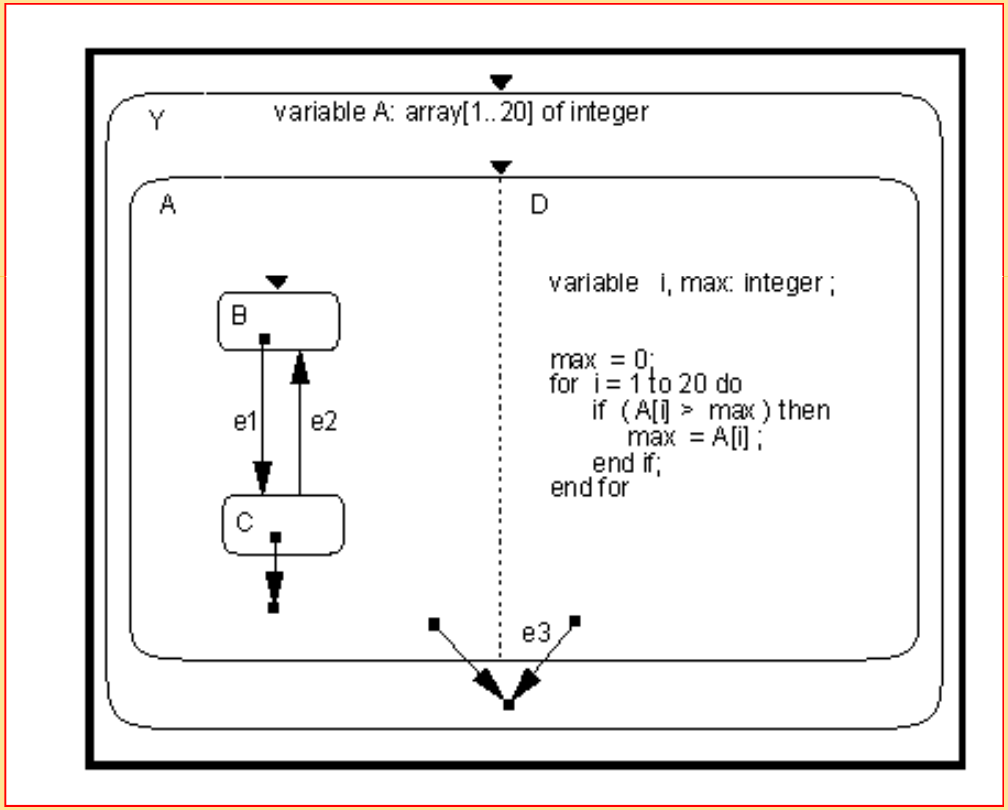
Heterogeneous: Program-state machine

Represent system's states, data, control and activities in a single model overcome the limitations of programming languages and HCFSM models

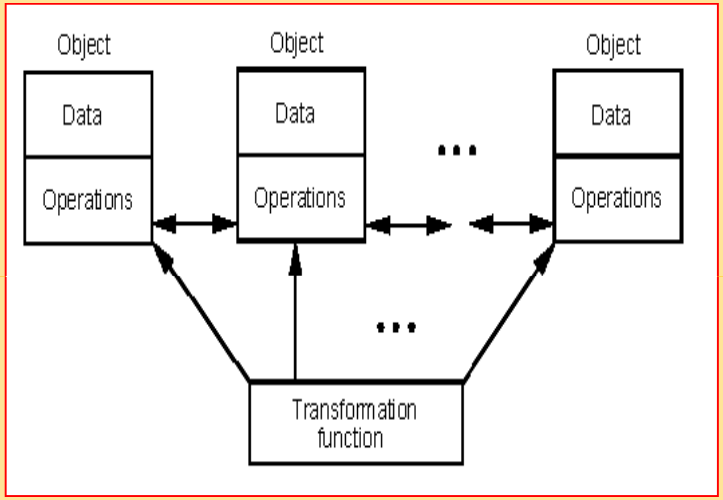


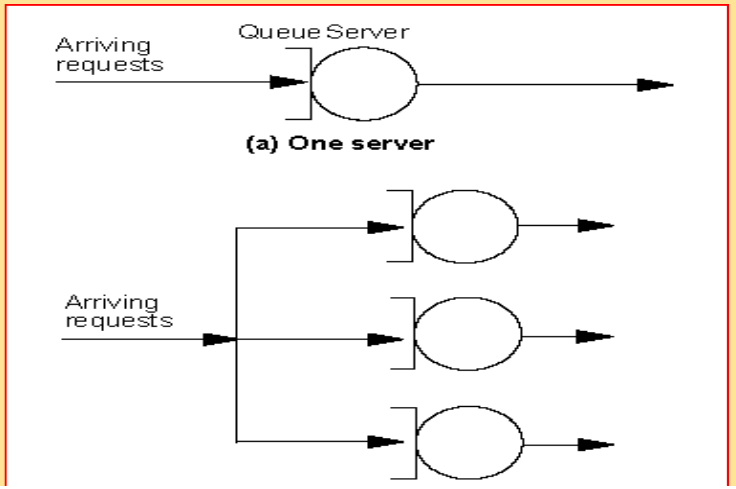
Object Oriented Model

Represent system's states, data, control and activities in a single model
overcome the limitations of programming languages and HCFSM models



Program-state machine



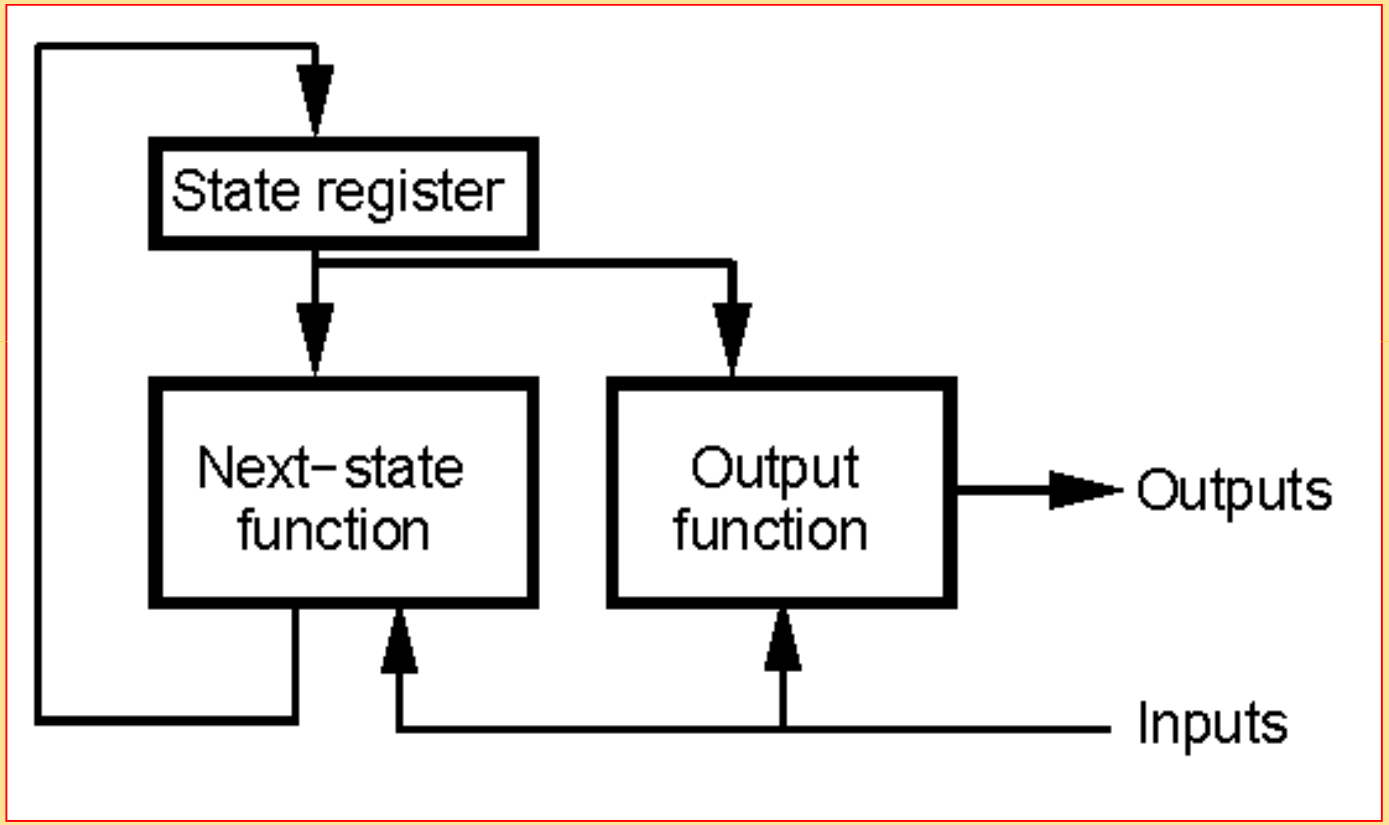


Architectures

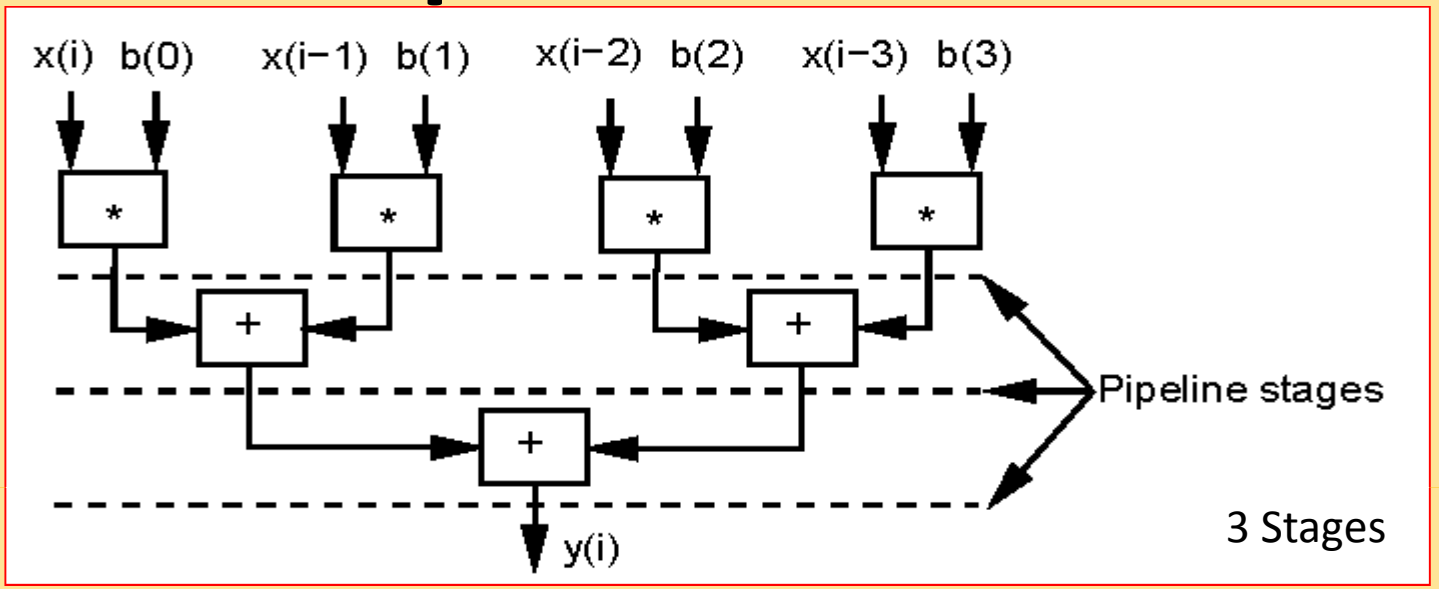
Architectures

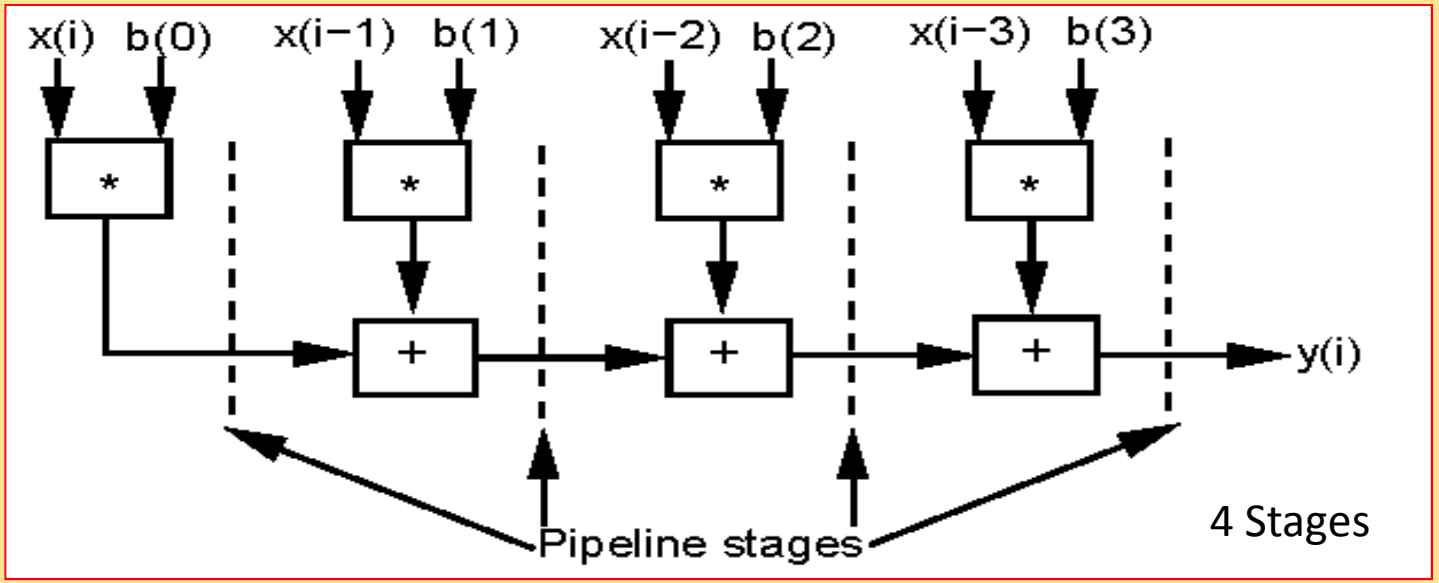
- Application-specific architectures
  - Controller architecture,
  - Datapath architecture,
  - Finite-state machine with datapath (FSMD)
- General-purpose processors
  - Complex instruction set computer (CISC)
  - Reduced instruction set computer (RISC)
  - Vector machine
  - Very long instruction word computer (VLIW)
- Parallel processors

Controller architecture

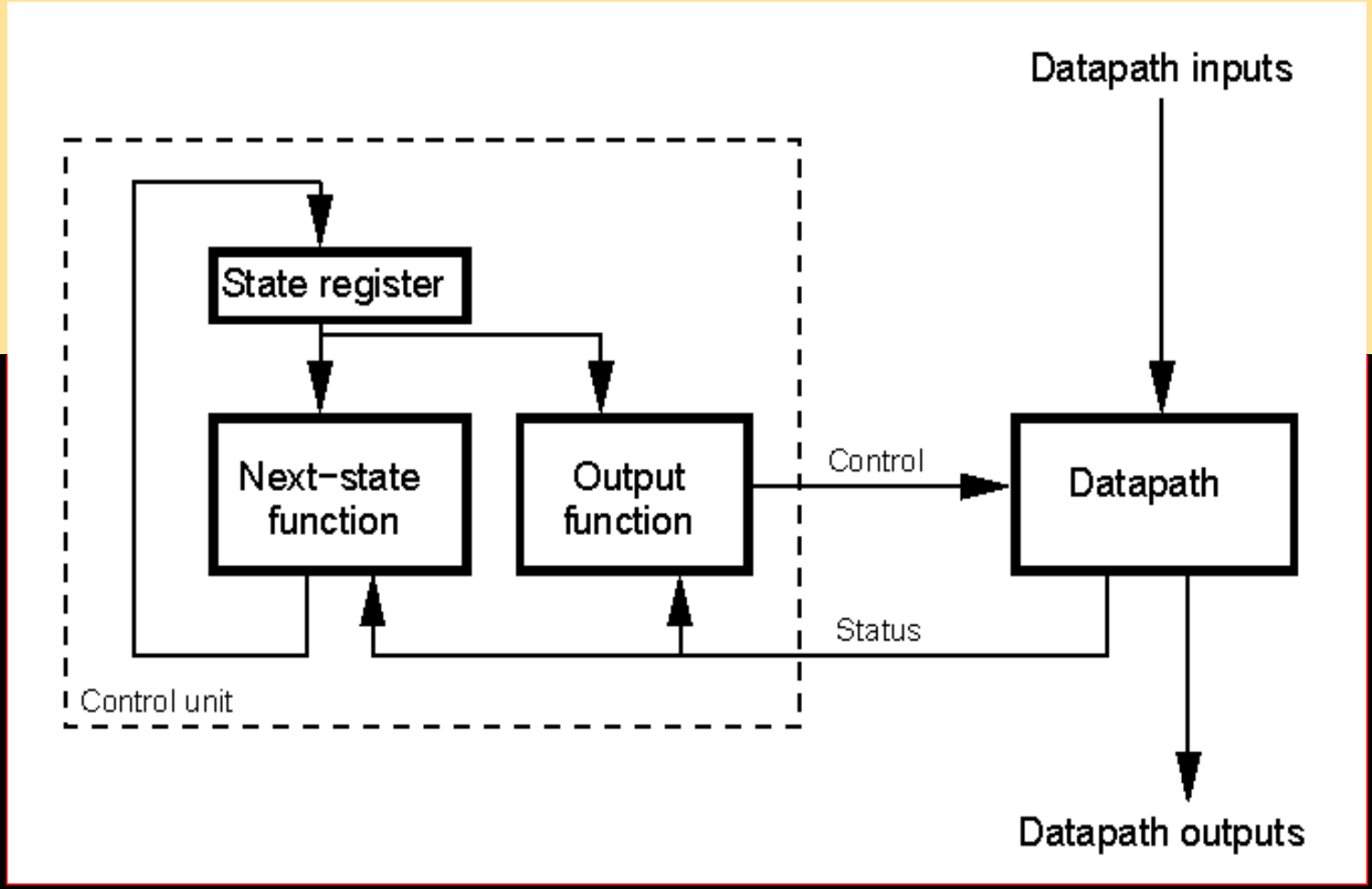


Datapath architecture

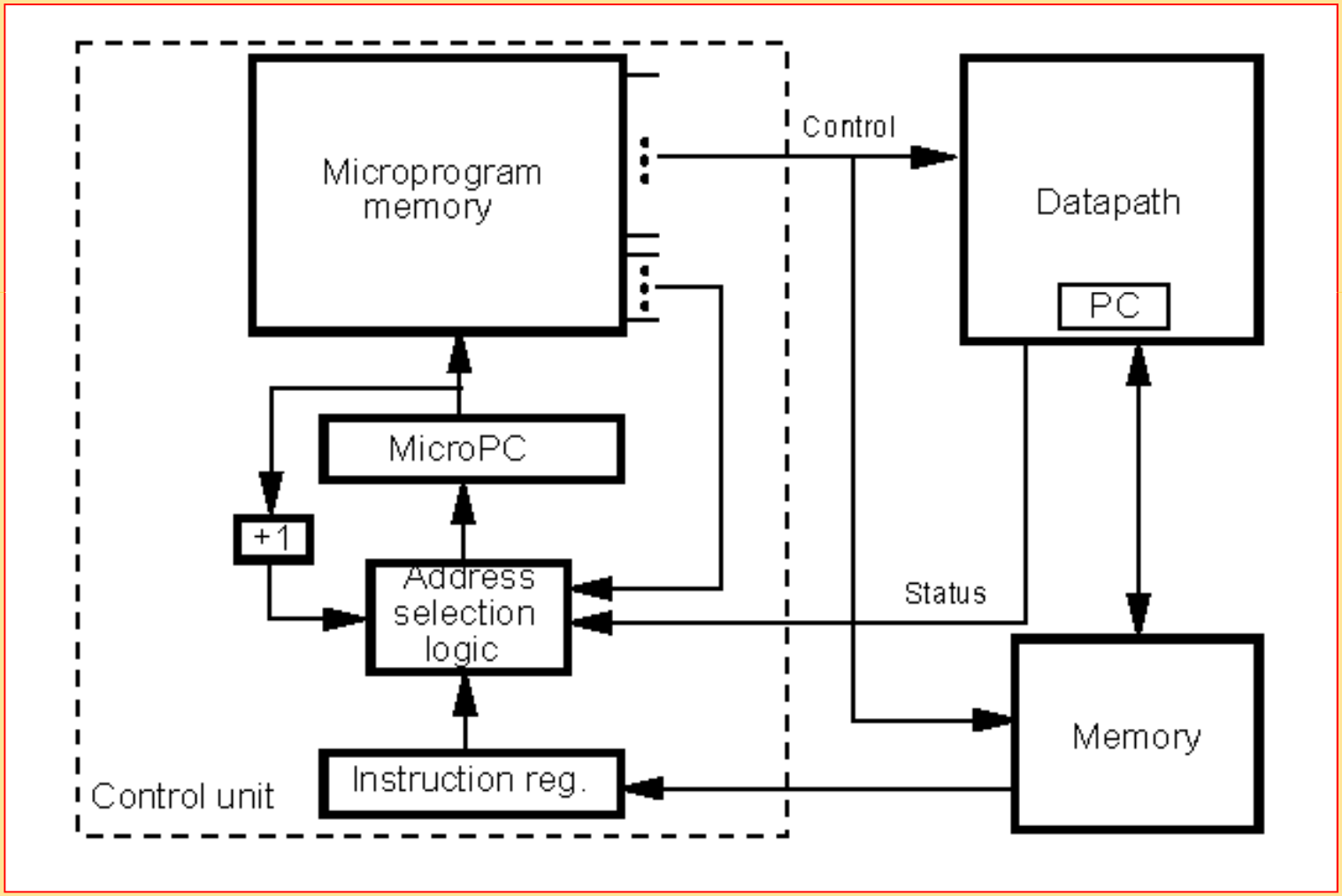




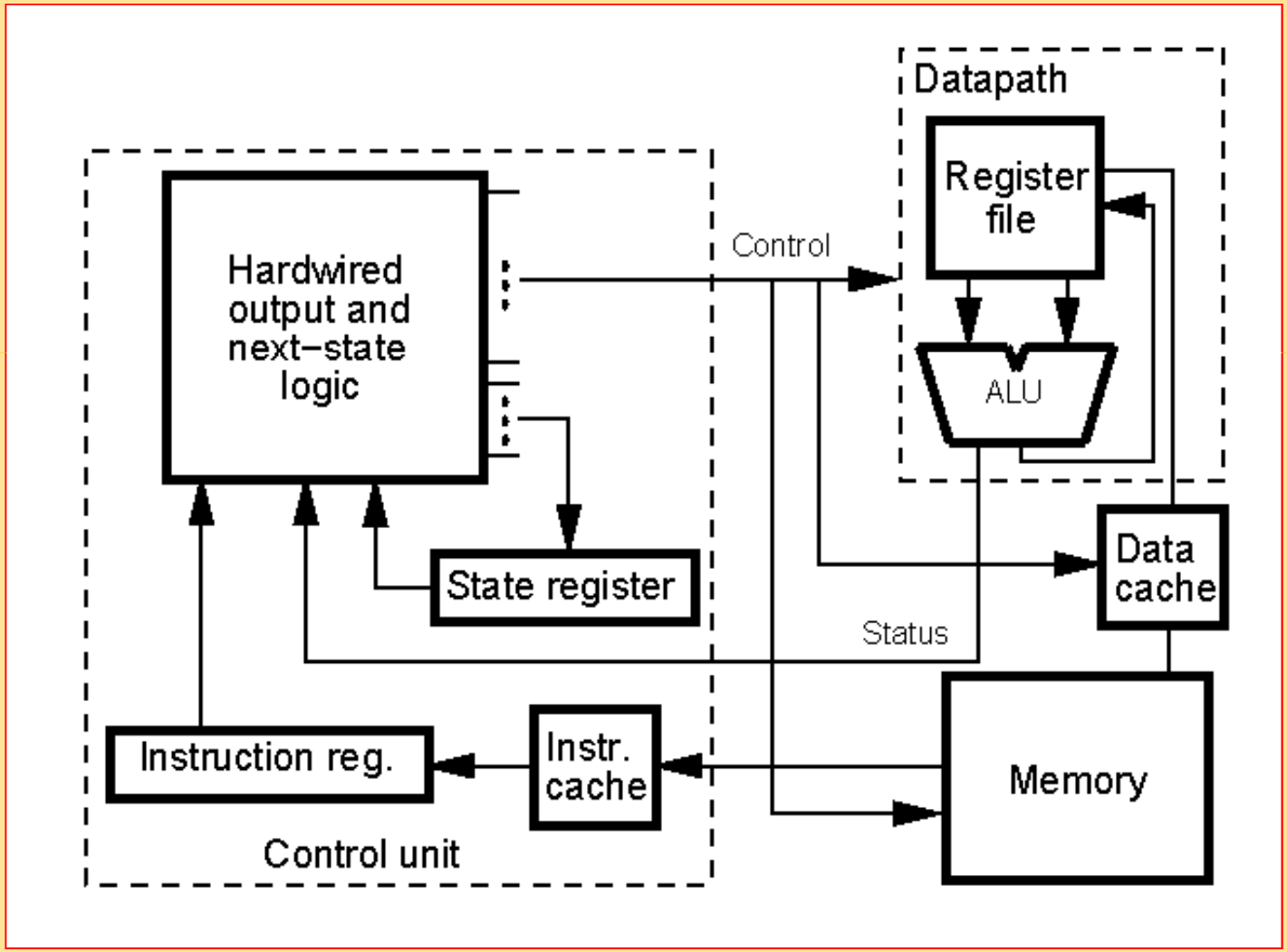
FSMD


Datapath inputs
State register
Next-state function
Output function
Control
Datapath
Status
Control unit
Datapath outputs

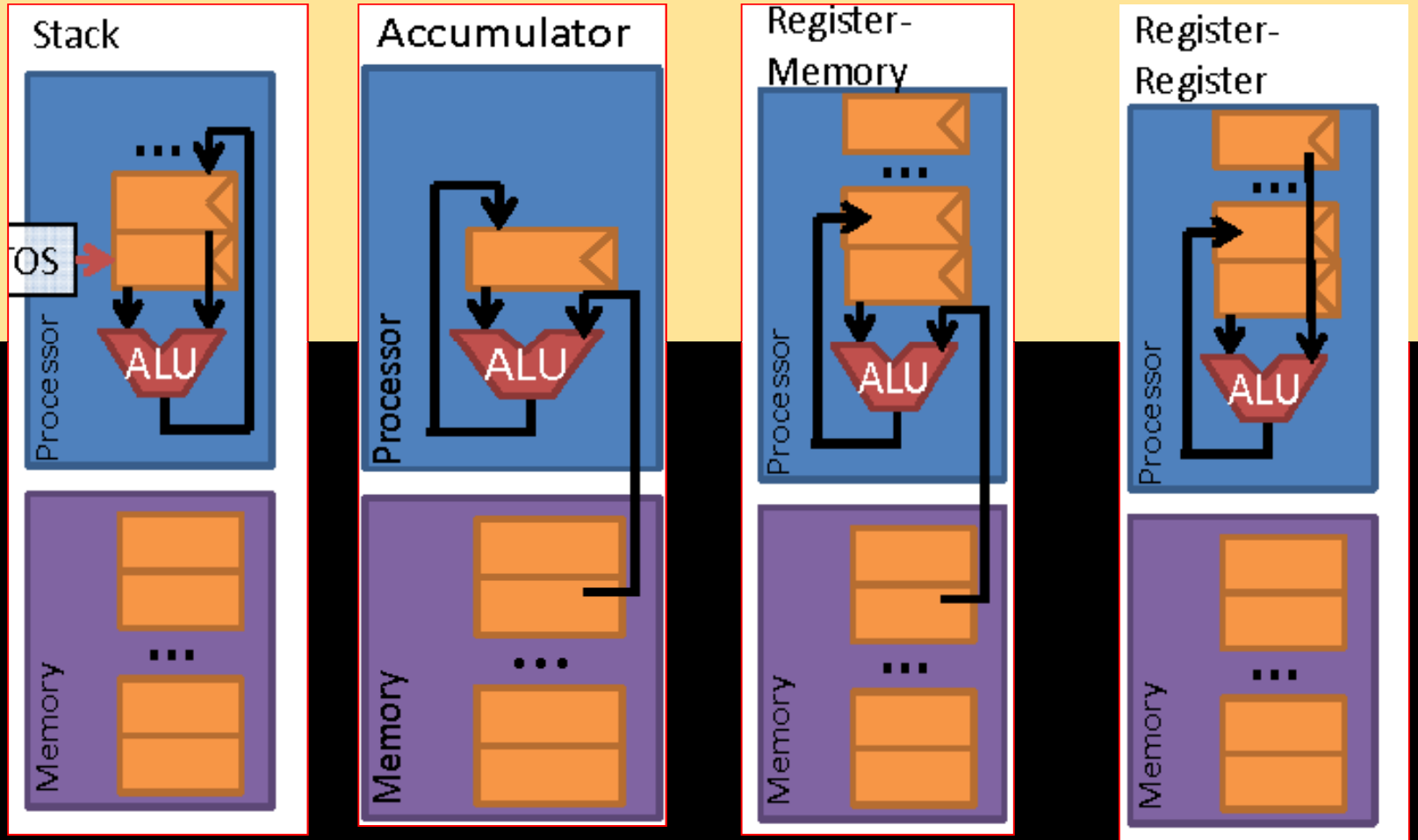
CISC Architecture



RISC Architecture



Vector Machine



Conceptual models

@172.16.1.72

- Data driven concurrency